

SUPSI

Valutazione di WebGPU per applicazioni di Realtà Virtuale tramite integrazione con OpenXR/OpenVR

Studente/i

Mazza Luca

Relatore

Peternier Achille

Correlatore

Paoliello Marco

Committente

Peternier Achille

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11287

Anno

2025 / 2026

Data

21 giugno 2026

Vorrei qui ringraziare

Indice

1	Abstract	1
2	Introduzione	3
2.1	Pianificazione	4
2.2	Requisiti	5
3	Stato dell'arte	7
3.1	Standard didattico - OpenGL & OpenVR	8
3.2	Frammentazione - Vulkan, DirectX & Metal	8
3.3	Comparativa - OpenVR & OpenXR	9
3.4	L'emergere di WebGPU	10
3.5	Conclusione	10
4	Design e implementazione	11
4.1	WebGPU	12
4.1.1	Architettura	12
4.1.2	Inizializzazione contesto & Windowing System	13
4.1.3	Gestione Memoria e assets	15
4.1.4	Pipeline e Bind Group	15
4.1.5	Command Encoder e Render Loop	16
4.2	Interoperabilità WebGPU-OpenXR	16
4.2.1	Rust <i>FFI Injection</i>	16
4.2.2	<i>Pointer Punning</i> per l'estrazione di memoria	17
4.2.3	Giustificazione architetturale	17
5	Test e validazione	19
5.1	Risultati	19
6	Risultati	20
6.1	Manuale d'uso	20
7	Conclusioni	21
7.1	Sviluppi futuri	21
	Glossario	I
	Sitografia	III

Elenco delle figure

3.1	OpenXR vs OpenVR (Khronos Group)	10
4.1	Shader class	12
4.2	WgpuBuffer class	12

Elenco delle tabelle

2.1	Requisiti del progetto	5
-----	----------------------------------	---

Elenco dei codici sorgente

1	Inizializzazione contesto finestra (Windows)	13
2	Inizializzazione contesto finestra (Linux Wayland/X11)	13
3	Inizializzazione contesto finestra (macOS)	14
4	Generazione dei dati texture con padding	15
5	Esempio di allineamento a 16 bytes	15
6	Esempio di FFI in Rust	17
7	Estrazione di una VkImage	17

Capitolo 1

Abstract

Capitolo 2

Introduzione

Negli ultimi anni, l'evoluzione delle API grafiche ha portato ad un progressivo allontanamento dagli standard tradizionali, come OpenGL¹, in favore di soluzioni più moderne, che forniscono più controllo direttamente sull'hardware delle unità grafiche dei calcolatori. In questo panorama, WebGPU² si è imposta non solo come nuova generazione di API grafiche per il Web, ma anche come una solida alternativa per contesti nativi, grazie ai kit di sviluppo che la rendono compatibile con diversi linguaggi a basso livello, come C++ e Rust. Un progetto precedentemente sviluppato ha già analizzato e dimostrato l'efficacia della transizione da OpenGL a WebGPU per corsi introduttivi alla computer grafica.

Lo scopo del presente progetto è di estendere tale studio di fattibilità in un settore con ancora maggiori esigenze, ovvero il dominio della *Realtà Virtuale* (VR). L'obiettivo principale è la valutazione dell'impiego di WebGPU in un ambiente nativo C/C++, e che questo possa rappresentare una valida alternativa ad OpenGL per lo sviluppo di applicazioni VR, investigando la complessità e l'efficacia della sua integrazione con framework standard quali OpenXR³ e OpenVR⁴.

Come per quanto riguarda i progetti precedentemente sviluppati, per valutare correttamente questa tecnologia è fondamentale tener sempre presente il contesto accademico in cui si andrà ad operare. Questo lavoro si rivolge in primis al modulo opzionale M-I6331Z - *Realtà Virtuale* della SUPSI, che si basa sulle conoscenze acquisite nel corso obbligatorio M-I5203 - *Grafica*. Lo scopo di tale corso è di insegnare allo studente lo sviluppo di un'applicazione interattiva in C++, interfacciandosi direttamente con le API di basso livello, senza affidarsi a game engine preconfezionati.

In questo scenario, la semplicità d'utilizzo è cruciale nel determinare la fattibilità; una soluzione troppo complessa rischierebbe di spostare inavvertitamente il focus del corso. Gli studenti e il docente si ritroverebbero a dedicare troppo tempo a districarsi tra le difficoltà di implementazione dell'API grafica, sottraendolo all'apprendimento dei concetti fondamentali della *Realtà Virtuale*. Tuttavia, lo sviluppo VR impone requisiti stringenti in termini di latenza, prestazioni e gestione diretta dell'hardware, elementi dove le API moderne come WebGPU eccellono, grazie al loro paradigma basato su pipeline esplicite.

Durante il progetto verrà pertanto sviluppato un prototipo funzionante che utilizzi WebGPU come backend grafico per il rendering di una scena VR stereoscopica. Verrà creato un layer di integrazione tra WebGPU e le API VR, affrontando e risolvendo le sfide tecniche legate all'acquisizione delle pose dei dispositivi, alla gestione delle swapchains e alla rigorosa sincronizzazione dei frame richiesta dal runtime, come ad esempio SteamVR.

¹<https://opengl.org>

²<https://webgpu.org>

³<https://khronos.org/openxr>

⁴<https://github.com/ValveSoftware/openvr>

L'esito di questa implementazione permetterà infine di confrontare l'approccio WebGPU con le soluzioni tradizionali, misurando in modo oggettivo il livello di effort richiesto per lo sviluppo e le performance ottenute. I risultati raccolti forniranno delle linee guida concrete per determinare l'effettiva fattibilità e le modalità ottimali per l'adozione di WebGPU in ambito didattico e applicativo nel contesto della Realtà Virtuale.

Con questo documento non si intende però produrre una guida dettagliata per quanto riguarda l'API di WebGPU, e nemmeno all'utilizzo di OpenXR o alle funzionalità VR, bensì saranno espliciti solamente i concetti essenziali alla comprensione di questo lavoro. Se si desidera apprendere i concetti riguardanti entrambe le API menzionate si può fare riferimento alla guida per i fondamentali di WebGPU[1] e a quella di OpenXR[2].

È anche da notare che questo documento non ha lo scopo di rendere noto ogni singolo dettaglio di codice sviluppato, bensì di accompagnare alla comprensione del problema, dei passi intrapresi per cercare di risolverlo e, se finalmente è fattibile sostituire l'approccio tradizionale con WebGPU.

2.1 Pianificazione

Di seguito sono descritte le differenti fasi atte allo sviluppo del progetto. In quanto la natura del progetto sia sperimentale, ed infine l'obiettivo sia quello di verificare la fattibilità di utilizzo, il piano di lavoro non eccede nei dettagli o nei vincoli ma risulta completo, garantendo la flessibilità necessaria e l'immediata risposta ad eventuali problemi o deviazioni dai mezzi discussi inizialmente.

1. **Analisi WebGPU:** Analizzare l'approccio necessario per lo sviluppo di un'applicazione grafica facente uso dell'API di WebGPU.
2. **Analisi OpenXR:** Analizzare l'approccio utilizzato nei progetti precedenti (specialmente per quanto riguarda *XRBridge*⁵) per lo sviluppo di un'applicazione grafica che faccia utilizzo di OpenXR per fornire funzionalità per la realtà virtuale.
3. **Demo WebGPU indipendente:** Derivante dall'analisi riguardante l'API di WebGPU, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità dell'API WebGPU. Questa demo deve rimanere semplice e ridurre la complessità di integrazione di ulteriori funzionalità, specialmente quelle riguardanti la realtà virtuale.
4. **Demo OpenXR indipendente:** Derivante dall'analisi riguardante OpenXR, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità di realtà virtuale. Questa demo deve essere imperativamente concentrata sui requisiti del corso di realtà virtuale e concentrarsi ad adattare unicamente ciò che è necessario per il corso. Ulteriormente, questa deve essere integrabile nell'ambiente WebGPU, deve quindi avere un approccio modulare, il quale permetterà di rimuovere le funzionalità OpenGL e sostituirle con quelle di WebGPU.
5. **Swapchain condivisa:** Sviluppare una swapchain, condivisa tra WebGPU e OpenXR, che permetta di condividere le texture da mostrare tramite gli occhi dell'headset, tra l'API WebGPU e quella di OpenXR. Ciò si riduce nella produzione di una libreria bridge, che utilizzi uno dei "minimi comun denominatori"⁶ tra le due tecnologie, in questo caso una tra Vulkan, DirectX o Metal.

⁵<https://github.com/JollyCookie2000/xr-bridge>

⁶WebGPU si interfaccia direttamente con l'API di sistema per la rappresentazione nativa di immagini grafiche.

6. **Sincronizzazione & Rendering Stereoscopico:** Integrare la sincronizzazione del loop di rendering di WebGPU con i tempi dettati da OpenXR. Deve essere inoltre modificata la pipeline di WebGPU per renderizzare la scena due volte, basandosi sulle matrici di *projection* e *view* fornite da OpenXR per occhio sinistro e destro.
7. **Demo unificata:** Produrre una demo che unisca tutte le funzionalità richieste dal corso di Realtà Virtuale, utilizzando l'architettura prodotta nelle fasi precedenti. Questa deve dimostrare in maniera semplice e efficace i risultati ottenibili dalla combinazione di WebGPU e OpenXR.
8. **Benchmarking:** Misurare il Framerate (FPS) e la latenza della demo, utilizzando appropriati strumenti di misurazione.
9. **Valutazione effort:** Confrontare la pipeline prodotta con quelle utilizzate negli anni passati del corso; valutare le principali differenze e le difficoltà di comprensione, la verbosità e l'ostacolo di debug.
10. **Linee guida conclusive:** Produrre una guida finale che mostri brevemente le conclusioni tratte durante lo sviluppo e che guidi l'eventuale integrazione della pipeline nel corso.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

Tabella 2.1: Requisiti del progetto

ID	Descrizione	Note
R-00	Deve presentare un backend WebGPU, con <code>wgpu_native</code> , in C++	< C++20
R-01	Deve presentare l'integrazione di OpenXR per gestire funzionalità VR/XR	-
R-02	Deve acquisire i dati di tracciamento dell'HMD	-
R-03	Deve gestire allocazione texture e submission duale con swapchains	-
R-04	Deve gestire la sincronizzazione con il framerate runtime VR	-
R-05	Deve presentare una demo completa di scena 3D, che utilizzi WebGPU	-
R-06	Deve essere compatibile con Windows e Linux	-
R-07	Deve includere analisi complessità, comparata alla soluzione OpenGL	-
R-08	Deve includere un'analisi della performance (benchmark)	-
R-09	Deve includere un verdetto finale e guida all'utilizzo	-

Capitolo 3

Stato dell'arte

Negli ultimi dieci anni, l'ecosistema dello sviluppo di applicazioni grafiche interattive ha visto intercorrere una trasformazione radicale. L'industria si è pian piano allontanata dai paradigmi tradizionali, basati su macchine a stati globali, per abbracciare interfacce di programmazione dal carattere esplicito, progettate per rispecchiare fedelmente l'architettura delle GPU multi-core. Ciò ha portato le API, contrariamente al trend seguito dalla maggior parte dell'ingegneria del software che si muove verso API di alto livello e astrazioni più vicine allo sviluppatore, a scendere sempre più di livello, avvicinandosi alla programmazione diretta dell'hardware grafico.

Parallelamente, il settore della Realtà Virtuale e Aumentata, comunemente raggruppato sotto il termine *Spatial computing* o XR, è maturato, passando da prototipi frammentari guidati da singoli vendor a standard aperti e unificati. Soprattutto per questo settore, il mercato ha avuto un moderato picco durante il periodo tra il 2020 ed il 2021, il quale ha poi visto un rapido declino, dovuto a diversi fattori, ma principalmente a scelte e underperformance dei produttori; specialmente lo spostamento dei settori di ricerca e sviluppo dei maggiori esponenti, come Microsoft e Meta, nel settore dell'Intelligenza Artificiale, in maniera affrettata (e poco considerata, visti i risultati ad oggi) ha posto fine all'evoluzione dei prodotti commerciali. Anche se il rilascio di un nuovo prodotto Apple nel settore, il prezzo imposto dalla casa di Cupertino non ha aiutato il mercato a rendere la tecnologia attrattiva.

Questo capitolo analizza le tecnologie attualmente impiegate in ambito didattico e industriale, evidenziando la crescente trasformazione dell'ecosistema grafico, le limitazioni e "l'invecchiamento" degli approcci tradizionali e la transizione architetturale verso nuovi standard aperti, come OpenXR e WebGPU.

3.1 Standard didattico - OpenGL & OpenVR

Nel contesto accademico e della formazione in computer grafica, il corso di Realtà Virtuale ha fatto affidamento sull'utilizzo di OpenGL per il rendering e OpenVR per l'interfacciamento con gli HMD.

OpenGL¹, introdotto nel 1992, è un'API implicita basata su una macchina a stati finiti. Il suo successo decennale, specialmente nell'ambito accademico, deriva dalla curva di apprendimento che risulta relativamente dolce; l'astrazione di alto livello utilizzata dagli sviluppatori risulta semplice, in quanto nasconde al programmatore la complessa gestione della memoria video, la sincronizzazione dei thread e la sottomissione dei comandi alla GPU. Tuttavia, questa astrazione ha un costo architetturale severo: i driver di OpenGL devono eseguire una massiccia mole di euristiche in background per tradurre le chiamate high level in istruzioni comprensibili per la scheda grafica, causando un notevole driver overhead e un'imprevedibilità delle prestazioni, soprattutto del frame stuttering, difetti critici nell'ambito della realtà virtuale, dove la latenza *motion-to-photon* deve rimanere strettamente inferiore ai 20 millisecondi.

OpenVR², sviluppato da Valve come SDK per la piattaforma SteamVR, è stata la prima API di successo commerciale a standardizzare l'accesso a device dedicati alla realtà virtuale *PC-VR*. Nata in un periodo di forte sperimentazione, questa tecnologia è intrinsecamente legata all'ecosistema di Valve, soprattutto con i visori di produzione HTC, per i quali le due aziende hanno collaborato nella produzione. Si basa su un modello nel quale l'hardware sta al centro dell'attenzione; infatti il runtime costringe l'hardware concorrente a tradurre i propri dati per renderli compatibili con il formato di un HTC Vive³, per esempio. Questa soluzione costringerebbe il programmatore a rilasciare una nuova patch ogni volta che un nuovo controller, visore o elemento periferico per il sistema VR viene rilasciato. Pur offrendo un'integrazione diretta e robusta con OpenGL, al momento si tratta di una tecnologia legacy, non più allineata con la traiettoria dell'industria moderna.

3.2 Frammentazione - Vulkan, DirectX & Metal

Il limite fisiologico di OpenGL ha portato alla nascita di una nuova generazione di API grafiche di basso livello; il successore naturale è **Vulkan**, sviluppato dallo stesso Khronos Group, **DirectX**, sviluppato da Microsoft con l'obiettivo di spingere il Gaming su PC, e **Metal**, di produzione Apple, per tutte le piattaforme correntemente sul mercato (iOS, macOS, tvOS, ecc...). Sebbene queste API garantiscano prestazioni eccezionali eliminando l'overhead dei driver e permettendo il multithreading nativo, hanno frammentato drasticamente il mercato, rendendo lo sviluppo multipiattaforma un'impresa titanica.

Vi sono due fattori critici che rendono queste API problematiche per la didattica:

1. Vulkan richiede un approccio esplicito alla gestione delle risorse. Il developer deve allocare manualmente la memoria, gestire i descriptor set, configurare le pipeline memory barrier e orchestrare le code di comandi. Realizzare il rendering di un singolo triangolo colorato utilizzando Vulkan richiede la stesura di 1000-1500 righe di codice, che a confronto delle 50-100 di OpenGL risultano eccessive per un corso di dieci settimane. Inoltre, un corso accademico focalizzato sulla realtà virtuale, dove gli argomenti principali dovrebbero essere la stereoscopia, l'interazione spaziale e la cinematica, forzare gli studenti ad apprendere quest'API significherebbe consumare l'intero semestre solo per inizializzare l'ambiente grafico.

¹<https://opengl.org>

²<https://github.com/ValveSoftware/openvr>

³<https://vive.com>

2. Apple ha ufficialmente deprecato OpenGL sui propri sistemi operativi, per poi rimuovere completamente il supporto nativo a tutte quelle tecnologie non-proprietarie che compongono lo standard *de-facto* dell'industria. Attualmente infatti, l'unica API grafica nativa supportata dall'hardware Apple Silicon e dai sistemi operativi moderni della casa californiana, è **Metal**. Apple ha rifiutato di adottare Vulkan, costringendo gli sviluppatori ad affidarsi a complessi layer di traduzione, come *MoltenVK*, per ottenere una parvenza di compatibilità. Questa tendenza a murare il proprio giardino distrugge il paradigma *Write once, Compile everywhere*, rendendo impossibile fornire un ambiente di sviluppo nativo e condiviso tra chi sceglie di utilizzare Windows, Linux o macOS.

3.3 Comparativa - OpenVR & OpenXR

La medesima frammentazione vissuta nel mercato delle API si è ripercossa su quello dell'hardware VR. Per risolvere questo problema, il *Khronos Group*⁴ ha rilasciato OpenXR, l'attuale standard industriale open-source per lo spatial computing, adottato universalmente da tutti i principali produttori, inclusa Valve stessa, che ha attivamente incoraggiato l'abbandono di OpenVR⁵.

Le differenze architetturali tra le due API sono profonde e rappresentano una rivoluzione concettuale nel modo in cui vengono concepite le applicazioni immersive. Di seguito le principali divergenze

- **Gestione Input (Hardware-Centric/Action-Based):** in OpenVR, il programmatore interroga direttamente l'hardware; questo approccio lo obbliga a riscrivere il codice ogni volta che un nuovo visore o una nuova periferica viene rilasciata sul mercato. In risposta a questo problema, OpenXR introduce un sistema **Action-Based**. Qui, lo sviluppatore definisce in modo astratto delle azioni logiche, per poi passarle al Runtime, il quale si occupa di mapparle sull'hardware utilizzato correntemente dall'utente in base a profili di interazione. Ciò rende le applicazioni scritte oggi con hardware che verrà rilasciato anche nel lontano futuro, senza necessità di modifiche o patching.
- **Gestione Spazi/Tracking:** OpenVR restituisce le matrici di posizionamento (*Poses*) basate su un sistema di coordinate cablato. OpenXR invece introduce il concetto di *XrSpace*⁶, dove ogni elemento, che si tratti del visore, del controller o della zona di gioco, è codificato come "Spazio". Al suo interno, la posizione di un oggetto viene calcolata interrogando il runtime, chiedendogli di localizzare uno spazio rispetto ad un altro, provvedendo a fornire anche stime sulle velocità lineari e angolari basate su complessi algoritmi di predizione, oltre che la posizione degli oggetti.
- **Architettura Swapchain:** per quanto riguarda OpenVR, l'applicazione grafica ha il compito di generare le texture e di provvederle al visore, tramite la funzione *Submit*. Al contrario, per quanto riguarda OpenXR, l'architettura è pensata all'inverso; è il runtime a possedere e allocare le immagini della Swapchain, garantendo ottimizzazione per l'hardware. L'applicazione deve quindi implementare un *Graphic Binding* esplicito, richiedendo al runtime i puntatori di memoria per poi disegnarci sopra. L'inversione del controllo rende estremamente complessa l'integrazione di API grafiche non ufficialmente supportate.

⁴<https://khronos.org>

⁵<https://windowscentral.com/steamvr-gets-open-source-upgrade-valve-transitions-openxr-api>

⁶<https://registry.khronos.org/OpenXR/specs/1.1/man/html/XrSpace.html>

3.4 L'emergere di WebGPU

Per colmare il vuoto lasciato dalla deprecazione di OpenGL e dalla complessità proibitiva di Vulkan, il *World Wide Web Consortium* (W3C) ha sviluppato WebGPU. Nata originariamente per succedere a WebGL, per i browser web, questa tecnologia è progettata con lo scopo di esporre le capacità delle API moderne garantendo sicurezza, portabilità ed un livello di astrazione ergonomico per lo sviluppatore.

La vera rivoluzione per l'ambiente desktop, ed il fulcro tecnologico di questo progetto, è il porting nativo di questa API tramite implementazioni C/C++, come **wgpu-native** e **Dawn**, sviluppate rispettivamente da Mozilla, in Rust e da Google in C++. Queste librerie operano come *Render Hardware Interface* (RHI) universale; lo sviluppatore scrive il codice relativo alla propria applicazione grafica, utilizzando l'API WebGPU standard e scrive gli shader nel linguaggio WGSL (WebGPU Shading Language). Il backend esegue poi automaticamente la traduzione *just-in-time* e invia i comandi all'API nativa del sistema operativo, quindi Vulkan/DirectX per quanto riguarda Windows, Vulkan per quanto riguarda Linux ed infine Metal per macOS.

WebGPU nativo risolve quindi simultaneamente due problemi fondamentali: riduce il boilerplate necessario per controllare l'hardware grafico, sempre utilizzando le API grafiche più moderne, performanti, efficienti e ottimizzate, e garantisce una compatibilità cross-platform assoluta, aggirando le restrizioni e l'isolazionismo di Apple.

3.5 Conclusione

L'industria dello spatial computing è ormai stabilmente posizionata su OpenXR, per la logica XR.

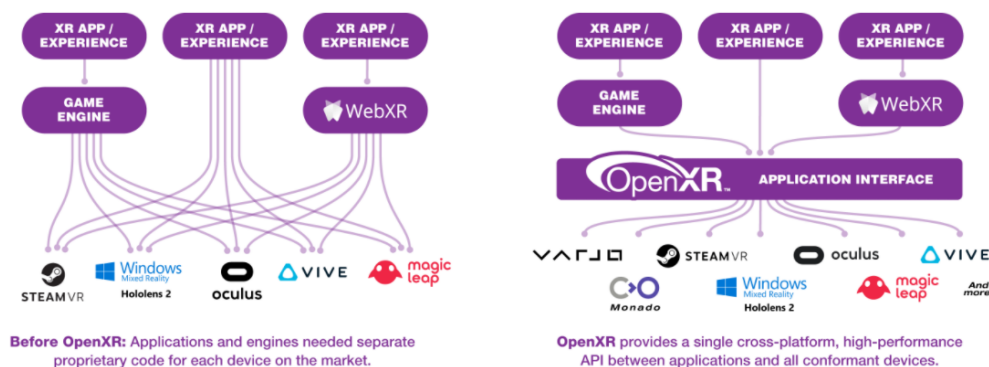


Figura 3.1: OpenXR vs OpenVR (Khronos Group)

Sul fronte grafico, mentre i motori commerciali (Unity, Unreal) o quelli proprietari, gestiscono le complesse API grafiche internamente, per lo sviluppo nativo ed educativo, WebGPU si sta affermando come soluzione più promettente.

Tuttavia, esiste attualmente un vuoto tecnologico per quanto riguarda l'API del W3C e OpenXR; tra le due tecnologie non esiste alcun Graphic Binding ufficiale da parte di Khronos. Quest'ultimo infatti definisce binding per OpenGL, Vulkan, Direct3D e Metal, ma non per WebGPU, essendo quest'ultima un'RHI e non consistente in un driver nativo.

Il presente lavoro si inserisce esattamente in questo contesto; l'obiettivo tecnico e scientifico è di dimostrare la fattibilità di un layer di interoperabilità che permetta di implementare nativamente con wgpu_native e che questa implementazione possa comunicare con la complessa swapchain ed il frame-pacing di OpenXR. Ciò permetterebbe di pensionare lo stack legacy e sostituirlo con uno all'avanguardia per il corso, con completa sostenibilità didattica e cross-platform per integrare i calcolatori di ogni studente.

Capitolo 4

Design e implementazione

Questo capitolo descrive in dettaglio la fase di implementazione del motore di rendering, che funge da *Proof of concept* per determinare la fattibilità dell'architettura sotto esame. L'implementazione viene affrontata in modo incrementale, data la mancanza di esperienza per quanto riguarda le tecnologie in questione, e viene divisa in tre pilastri fondamentali, che verranno esplorati nelle sezioni seguenti:

1. Il motore grafico di base WebGPU, consistente nella costruzione di una pipeline di rendering esplicita ed autonoma, utilizzando l'API `wgpu_native`. Questa fase risolve le critiche relative all'allineamento della memoria, alla gestione esplicita dello stato ed al caricamento degli assets.
2. L'integrazione di OpenXR, che vede l'implementazione di un'interfaccia indipendente dall'hardware per comunicare con i visori di realtà virtuale, gestendo il tracciamento spaziale, gli stati della sessione e l'acquisizione della swapchain.
3. Il ciclo applicativo principale che fa da ponte tra i due sottosistemi, sincronizzando il rendering stereoscopico di WebGPU con i requisiti temporali di visualizzazione del compositor OpenXR.

Analizzando queste tre fasi, questo capitolo illustra come i requisiti ingegneristici teorici vengono tradotti in un prototipo funzionale, ottimizzato e possibilmente multiplatforma.

4.1 WebGPU

La seguente sezione si concentra sulle funzionalità di WebGPU implementate, utili a raggiungere lo stesso risultato dimostrato nel tutorial ufficiale del corso di realtà virtuale.

È importante che la codebase, sebbene facente uso di un'API con struttura e filosofia radicalmente differenti, sia strutturata il più simile possibile a quella del corso, per rendere la comparazione il più semplice ed efficiente possibile.

Altresì importante è tenere a mente che il codice prodotto è pensato per uno studente, è quindi da considerare le piattaforme e il livello di confidenza nell'apprendere un'API del genere.

4.1.1 Architettura

A differenza di OpenGL, il quale mantiene uno stato globale e gestisce la sincronizzazione della memoria in background, la nuova architettura richiede definizioni esplicite per ogni singola fase della pipeline. Per mantenere una minima leggibilità e separare qualche responsabilità, pur lasciando la struttura simile a quella del tutorial del corso per facilitarne la comparazione; per questo, seguendo l'architettura di corso, sono state modularizzate le classi:

- **Shader**: Gestisce il caricamento, la compilazione ed il ciclo di vita degli Shader Modules WebGPU, partendo dalla sorgente di codice, nel linguaggio WGSL (Web GPU Shading Language). Evita il caricamento di file esterni direttamente dal punto d'entrata del motore.

Shader
-m_module: WGPUShaderModule
+Shader() +~Shader() +loadFromFile(WGPUDevice, char*): bool +destroy(): void +getModule(): WGPUShaderModule

Figura 4.1: Shader class

- **WgpuBuffer**: Incapsula la creazione e l'aggiornamento dei buffer GPU, detti `WGPUBuffer`, imponendo il rigoroso allineamento della memoria a 4 byte, richiesto dalle specifiche dell'API per i vertici ed i dati *uniform*.

WgpuBuffer
-m_buffer: WGPUBuffer -m_alignedSize: size_t
+WGPUBuffer() +~WGPUBuffer() +create(WGPUDevice, WGPUQueue, void*, size_t, WGPUBufferUsage): bool +update(WGPUQueue, void*, size_t, size_t): void +get(): WGPUBuffer +size(): size_t

Figura 4.2: WgpuBuffer class

Questo incapsulamento garantisce che il ciclo principale dell'applicazione, includendo i descrittori della pipeline e la codifica dei comandi, rimanga pulito e focalizzato esclusivamente sull'inizializzazione e l'orchestrazione.

4.1.2 Inizializzazione contesto & Windowing System

La fase di inizializzazione del contesto, richiede il collegamento dell'istanza WebGPU ad un sistema di windowing al livello del sistema operativo. Questo requisito risulta relativamente complesso per l'integrazione della nuova API, in quanto questa è pensata per il web, quindi non avendo alcun accesso diretto alle API di sistema per l'acquisizione della finestra.

Windows

Per i sistemi operativi della famiglia Windows, l'integrazione risulta relativamente lineare, grazie all'accesso diretto alle API Win32 fornite dalla libreria GLFW. Il descrittore specifico per questa piattaforma, `WGPUSurfaceSourceWindowsHWND`, richiede la provvigione di due parametri fondamentali: l'handle dell'istanza dell'applicazione (`hinstance`), recuperabile tramite la chiamata di sistema `GetModuleHandle`, e l'handle della finestra nativa `hwnd`. Come mostrato nel listing 1, questi dati vengono incapsulati e concatenati direttamente al descrittore generico della *Surface*, sfruttando il meccansimo del puntatore `nextInChain`.

```
WGPUSurfaceDescriptor surfaceDesc= {};
#ifdef _WIN32
WGPUSurfaceSourceWindowsHWND hwndDesc = {
    .chain = { .sType = WGPUType_SurfaceSourceWindowsHWND },
    .hinstance = GetModuleHandle(NULL),
    .hwnd = glfwGetWin32Window(window)
};
surfaceDesc.nextInChain = (WGPUChainedStruct*)&hwndDesc;
```

Listing 1: Inizializzazione contesto finestra (Windows)

Linux

Nel panorama Linux, l'acquisizione del contesto della finestra richiede un approccio dinamico a causa della coesistenza di due differenti protocolli per server grafici: l'ormai legacy *X11* ed il più moderno *Wayland*. Per garantire la massima portabilità e compatibilità, l'implementazione non assume a priori l'infrastruttura sottostante, ma interroga la variabile d'ambiente `XDG_SESSION_TYPE` del sistema operativo per determinare il compositor attualmente in uso. A seconda del risultato (vedi listing 2), l'algoritmo istanzia il descrittore appropriato estraendo da GLFW i rispettivi puntatori nativi per il display e per la superficie di rendering.

```
#elif defined (__linux__)
char *envSession = std::getenv("XDG_SESSION_TYPE");
std::string windowingSystem = envSession ? envSession : "x11";
if (windowingSystem == "x11") {
    WGPUSurfaceSourceXlibWindow x11Desc = {
        .chain = { .sType = WGPUType_SurfaceSourceXlibWindow },
        .display = glfwGetX11Display(),
        .window = glfwGetX11Window(window) };
    surfaceDesc.nextInChain = (WGPUChainedStruct*)&x11Desc;
} else if (windowingSystem == "wayland") {
    WGPUSurfaceSourceWaylandSurface waylandDesc = {
        .chain = { .sType = WGPUType_SurfaceSourceWaylandSurface },
        .display = glfwGetWaylandDisplay(),
        .surface = glfwGetWaylandWindow(window) };
    surfaceDesc.nextInChain = (WGPUChainedStruct*)&waylandDesc;
}
```

Listing 2: Inizializzazione contesto finestra (Linux Wayland/X11)

macOS

L'ecosistema Apple presenta il livello di complessità più alto per quanto riguarda la creazione della superficie, poiché su queste piattaforme WebGPU si appoggia nativamente al backend grafico Metal. Per evitare di dover introdurre nella codebase C++ delle sorgenti scritte in ObjectiveC, che richiederebbe l'uso di sorgenti .mm, l'implementazione sfrutta l'iniezione a runtime tramite la funzione `objc_msgSend` che permette di interagire direttamente con le librerie di sistema Cocoa. Il processo, consultabile nel listing 3, si occupa di estrarre dapprima l'oggetto `NSWindow`, ne recupera poi la vista principale `contentView` e vi aggancia dinamicamente un livello di rendering dedicato, chiamato `CAMetalLayer`. Inoltre, durante questa fase, viene esplicitamente disabilitata la proprietà `displaySyncEnabled` del layer Metal; questo è un passaggio fondamentale per aggirare la sincronizzazione verticale forzata in modo aggressivo dal *Window Server* di macOS, permettendo all'applicazione di sbloccare il framerate per una migliore profilazione delle performance. Il livello così configurato viene finalmente passato al descrittore `WGPUSurfaceSourceMetalLayer`.

```
#elif defined(__APPLE__)
#include <objc/message.h>
#include <objc/runtime.h>
id nsWindow = glfwGetCocoaWindow(window);
id nsView = ((id*)(id, SEL))objc_msgSend(nsWindow,
    sel_registerName("contentView"));
id caMetalLayerClass = (id)objc_getClass("CAMetalLayer");
id metalLayer = ((id*)(id, SEL))objc_msgSend(caMetalLayerClass,
    sel_registerName("layer"));
((void*)(id, SEL, bool))objc_msgSend(nsView,
    sel_registerName("setWantsLayer:"), true);
((void*)(id, SEL, id))objc_msgSend(nsView,
    sel_registerName("setLayer:"), metalLayer);
((void*)(id, SEL, bool))objc_msgSend(metalLayer,
    sel_registerName("setDisplaySyncEnabled:"), false);
WGPUSurfaceSourceMetalLayer metalDesc = {
    .chain = { .sType = WGPUType_SurfaceSourceMetalLayer },
    .layer = metalLayer
};
surfaceDesc.nextInChain = (WGPUChainedStruct*)&metalDesc;
#endif
```

Listing 3: Inizializzazione contesto finestra (macOS)

Una volta configurato il descrittore specifico per la piattaforma host corrente tramite la catena di strutture `nextInChain`, la superficie viene concretamente istanziata e restituita al ciclo di vita dell'applicazione tramite la chiamata nativa dell'API.

```
return wgpuInstanceCreateSurface(instance, &surfaceDesc);
```

4.1.3 Gestione Memoria e assets

Il passaggio dal `glTexImage2D` di OpenGL al sistema adottato da WebGPU richiede il calcolo manuale degli *stride* di memoria. La libreria `stb_image` è stata integrata per caricare gli assets direttamente in un formato *top-down*, corrispondendo nativamente al sistema di coordinate delle texture WebGPU ed eliminando la necessità di invertire l'asse *y* durante il caricamento.

L'API impone un vincolo sui caricamenti delle texture tramite `wgpuQueueWriteTexture`: il parametro `bytesPerRow` deve essere obbligatoriamente un multiplo di 256 bytes. Questo problema è stato risolto calcolando e applicando matematicamente un padding al buffer dei dati grezzi dell'immagine, prima di consegnarlo alla GPU.

```
uint32_t bytesPerRow = (texWidth * 4 + 255) & ~255;
std::vector<uint8_t> paddedData(bytesPerRow * texHeight, 0)
for (int y = 0; y < texHeight; ++y) {
    memcpy(&paddedData[y*bytesPerRow], &rawData[y*texWidth*4], texWidth*4);
}
```

Listing 4: Generazione dei dati texture con padding

Un altro fondamentale cambiamento di paradigma riguarda l'allineamento delle strutture tra C++ e WGSL. Questo infatti impone rigorose regole di layout nella memoria, nelle quali tipi di dati come `vec3` devono essere strettamente allineati a limiti di 16 bytes. Per prevenire la desincronizzazione della memoria tra le `struct` di C++ a 128 bytes e la `struct` WGSL attesa, da 144 bytes, è stato introdotto un padding manuale nelle strutture dati C++ per soddisfare le aspettative di memoria della GPU, senza sprecare larghezza di banda.

```
struct LightMaterialUniforms {
    glm::vec4 lightPosition;
    glm::vec4 lightAmbient;
    glm::vec4 lightDiffuse;
    glm::vec4 lightSpecular;
    glm::vec4 matAmbient;
    glm::vec4 matDiffuse;
    glm::vec4 matSpecular;
    float matShininess;
    float padding[3];
};
```

Listing 5: Esempio di allineamento a 16 bytes

4.1.4 Pipeline e Bind Group

La pipeline di rendering funge da oggetto di stato immutabile. Il prototipo utilizza un modello di illuminazione *Blinn-Phong*, proprio come nel tutorial del corso, il tutto tradotto in una *Shader* scritta nel linguaggio WGSL. Durante la creazione della pipeline, viene sfruttata la funzionalità di *Auto-Layout*.

```
pipelineDesc.layout = nullptr;
```

Questo istruisce il backend WebGPU a inferire dinamicamente il layout del Bind Group, direttamente dalle annotazioni del linguaggio di shading (`@group` e `@binding`), riducendo significativamente la verbosità del C++.

Le risorse da fornire al programma shader sono divise in due distinti Bind Group in base alla loro frequenza di aggiornamento:

1. **Bind Group 0:** contiene gli Uniform Buffer per le matrici *Model-View-Projection* ed i parametri dei materiali di illuminazione, aggiornati ad ogni frame
2. **Bind Group 1:** contiene il *Sampler* e la *Texture View*, che rimangono statici per tutto il ciclo di vita dell'applicazione

Inoltre, le rigide regole di validazione dell'API hanno richiesto una gestione esplicita delle *C Strings*. Con i recenti aggiornamenti delle API, gli entrypoint degli shader devono essere passati come oggetti di tipo `WGPUStringView`, che abbinano il contenuto della stringa con una macro di lunghezza specifica (`WGPU_STRLEN`), piuttosto che come puntatori terminati da carattere nullo, al fine di gestire la sicurezza della memoria attraverso i confini del backend Rust.

4.1.5 Command Encoder e Render Loop

Il ciclo principale vede un `WGPUCommandEncoder` che registra una serie di istruzioni all'interno di un command buffer, che viene successivamente sottomesso alla queue della GPU in una singola operazione.

Una cruciale implementazione di stabilità all'interno del ciclo di rendering è la gestione dello stato di acquisizione della `WGPUSurfaceTexture`. Le latenze del sistema operativo, come il ridimensionamento o la riduzione ad icona della finestra potrebbero far sì che la superficie "perda" temporaneamente accesso alla texture del frame corrente; l'aggiunta di una *guard clause* che valuti lo stato `WGPUSurfaceGetCurrentTextureStatus_Success` impedisce al backend Rust di andare in *panic* alla ricezione di un puntatore nullo, garantendo la totale robustezza del motore in caso di eventi esterni.

Infine, il `WGPURenderPassDescription` richiede definizioni esplicite e complete, inclusa la specifica del parametro `depthSlice` a `WGPU_DEPTH_SLICE_UNDEFINED`, per i color attachment 2D, una configurazione necessaria per conformarsi in modo rigoroso alle specifiche di rendering delle texture 3D, recentemente introdotte in WebGPU.

4.2 Interoperabilità WebGPU-OpenXR

L'integrazione tra WebGPU e OpenXR presenta una sfida architetturale fondamentale legata ai paradigmi di sicurezza ed astrazione delle due API; WebGPU, per sua natura, è agnostica e sicura: il suo obiettivo è di nascondere completamente i dettagli hardware sottostanti dietro ad un'interfaccia unificata. Al contrario, OpenXR per operare in ambiente VR richiede un'accesso esplicito e diretto ai puntatori nativi della GPU, ed alle allocazioni di memoria delle immagini per poter gestire la swapchain a bassa latenza.

Nella versione più recente¹, le funzioni di estrazione per il backend Vulkan sono state rimosse dall'API pubblica, sigillando di fatto i puntatori nativi all'interno dell'Hardware Abstraction Layer, scritto in Rust. Ciò rende impossibile l'inizializzazione standard di una `XrSession` basata sul contesto grafico di WebGPU.

4.2.1 Rust FFI Injection

Per aggirare questa limitazione senza dover riscrivere interi moduli della libreria si è optato per una tecnica di iniezione di codice, tramite una tecnica chiamata *Foreign Function Interface*, direttamente nel codice sorgente di `wgpu-native`.

Sfruttando le capacità di interoperabilità e esportazione in C di Rust, vengono iniettate delle funzioni custom, in grado di interrogare l'oggetto `context` all'interno di WebGPU.

¹<https://github.com/gfx-rs/wgpu-native/tree/v29.0.0.0>

Ciò permette di forzare l'astrazione e di risalire all'istanza Vulkan nativa, restituendola all'API C++ sottoforma di puntatore opaco.

```
#[no_mangle]
pub unsafe extern "C" fn function(...) -> *mut c_void {}
```

Listing 6: Esempio di FFI in Rust

4.2.2 Pointer Punning per l'estrazione di memoria

Il problema più critico si presenta quando si deve gestire la *Swapchain*. OpenXR richiede che i fotogrammi vengano renderizzati direttamente sulle sue *VkImage*. Tuttavia, le strutture interne di WebGPU possiedono campi strettamente privati, impedendo la costruzione o la manipolazione di texture a partire da memoria pre-allocata esternamente.

La soluzione sfrutta una tecnica di accesso diretto alla memoria, nota come *Pointer Punning*. Sapendo per costruzione che il primo campo della struttura *Texture* nel backend Vulkan di WebGPU corrisponde all'handle nativo `ash::vk::Image`, è possibile bypassare la protezione del compilatore castando direttamente l'indirizzo di memoria della struttura ad un puntatore Vulkan.

```
#[no_mangle]
pub unsafe extern "C" fn wgpuTextureGetVulkanImage(
    texture: crate::native::WGPUTexture,
) -> *mut c_void {
    #[cfg(all(
        any(target_os="windows", target_os="linux"),
        feature="vulkan"
    ))]
    {
        let texture = texture.as_ref().expect();
        let hal_tex =
            texture.context.texture_as_hal::<hal::api::Vulkan>(texture.id);
        if let Some(hal_device) = hal_device {
            return &hal_device.raw_device().handle() as *const _ as *mut c_void;
        }
        std::ptr::null_mut()
    } // Fallback per altri OS
}
```

Listing 7: Estrazione di una *VkImage*

4.2.3 Giustificazione architetturale

Inizialmente si era valutata l'ipotesi di istanziare un oggetto *WGPUTexture* direttamente sopra la memoria allocata da OpenXR, tecnica detta *Memory Aliasing*. Questo approccio è stato tuttavia scartato, in favore dell'estrazione diretta della *VkImage* descritta nel Listing 7, per due ragioni fondamentali legate alla sicurezza del ciclo di vita della memoria, il quale è fondamentale per il corretto funzionamento soprattutto in una codebase Rust.

1. Se le due API condividessero lo stesso blocco di memoria, la distruzione al termine del programma genererebbe conflitti fatali, poiché entrambi i sistemi tenterebbero di deallocare la stessa risorsa. Questa situazione è anche nota come *Double-Free*.
2. L'architettura implementata separa le responsabilità; WebGPU renderizza sulle proprie textures, mentre OpenXR alloca la sua *Swapchain*. Sfruttando la funzione nel Listing 7, il motore C++ può eseguire una copia diretta dei pixel ad un bassissimo livello di Vulkan; quest'ultima tecnica è detta *Command Copying* o *Blitting*, e si può eseguire tramite la funzione `vkCmdCopyImage`.

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

DirectX (in origine chiamato "Game SDK") è una raccolta di API per lo sviluppo semplificato di videogiochi per sistema operativo Microsoft Windows. Il componente DirectX Graphics permette la presentazione di grafica 2D e 3D a video, direttamente interfacciandosi con la GPU. i, 4, 8

Foreign Function Interface Meccanismo mediante il quale un programma scritto in un linguaggio di programmazione può chiamare routine o fare uso di servizi scritti in un altro. 16

Framerate (FPS) Frequenza di cattura o riproduzione dei fotogrammi che compongono un filmato. Un filmato, o un'animazione al computer, è infatti una sequenza di immagini riprodotte ad una velocità sufficientemente alta da fornire, all'occhio umano, l'illusione del movimento. 5

HMD Head-Mounted Display (in italiano schermo montato sulla testa), schermo montato sulla testa dello spettatore attraverso un casco ad-hoc e può essere monoculare o binoculare. 5

Metal Insieme di API grafiche e di calcolo a basso livello, sviluppate da Apple per offrire accesso diretto alla GPU dei suoi dispositivi. i, 4, 8–10, 14

nextInChain Convenzione tipica delle moderne API grafiche derivate da Vulkan, per l'estensione dinamica delle strutture dati. 13, 14

OpenVR API e Runtime che consente accesso alle risorse hardware VR di diversi produttori, senza richiedere conoscenze specifiche dell'hardware stesso. 3

OpenXR Standard aperto che fornisce un set di API per lo sviluppo di applicazioni XR (realtà virtuale, mista, ecc...) compatibili con un grande range di dispositivi AR e VR. i, 3–5, 7, 9–11, 16, 17

Swapchain Serie di framebuffer virtuali utilizzati dalla scheda grafica e dall'API grafica per la stabilizzazione del frame rate, la riduzione delle interruzioni e diversi altri scopi. 17

Vulkan Interfaccia programmatica di applicazione (API) di basso livello, multi-piattaforma in 2D e 3D, annunciata la prima volta al GDC 2015 da Khronos Group. Inizialmente venne presentata come "OpenGL di prossima generazione". i, 4, 8–10, 16, 17

Sitografia

- [1] Gregg Travers. *WebGPU Fundamentals*. 2020. URL: <https://webgpufundamentals.org> (visitato il 28/05/2026).
- [2] The Khronos Group. *OpenXR Tutorial*. 2024. URL: <https://openxr-tutorial.com> (visitato il 28/05/2026).